

Java Collection Framework

2) Generic Collection :-

- Introduced in Java 1.5 for type safety.
- Can store only one type of data (homogeneous).
- No need for type casting.
- Errors detected at compile time.

Syntax :-

CollectionType<DataType> variableName = new CollectionType<DataType>();

Example :-

```
package collections;

import java.util.ArrayList;
import java.util.Iterator;

public class generic_collection {

    public static void main(String[] args) throws InterruptedException {
        ArrayList<String> name=new ArrayList<String>();
        name.add("Sagar");
        name.add("Rahul");
        name.add("Ganesh");

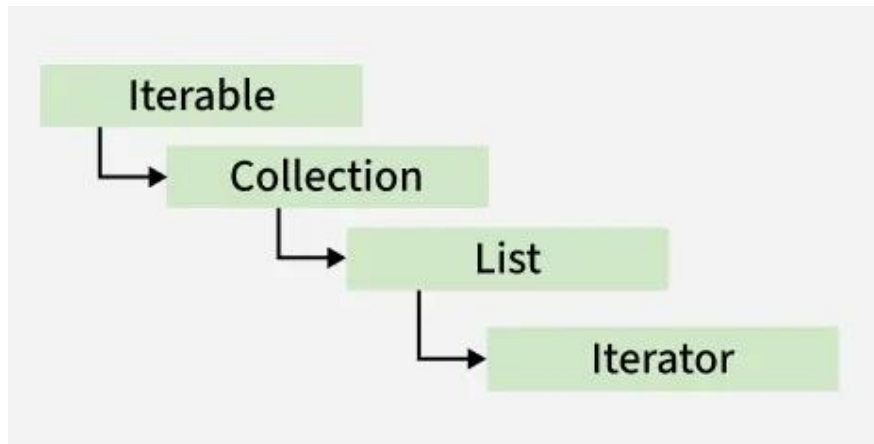
        System.out.println("Generic Collection is: "+name);
    }
}
```

Output :-

Generic Collection is: [Sagar, Rahul, Ganesh]

Iterator in Java :-

- Iterator is an interface in the Java Collection Framework.
- It is used to traverse elements one by one from a collection.
- It works with all collection classes like ArrayList, HashSet, etc.
- Iterator is unidirectional — it moves only in forward direction.



Methods of Iterator:-

Method	Description
boolean hasNext()	Checks if more elements are present in the collection.
Object next()	Returns the next element in the collection.
void remove()	Removes the last element returned by next() (optional).

Syntax :-

Iterator<DataType> it = collectionName.iterator();

Example :-

```
package collections;

import java.util.ArrayList;
import java.util.Iterator;

public class generic_collection {

    public static void main(String[] args) throws InterruptedException {
        ArrayList<String>name=new ArrayList<String>();
        name.add("Sagar");
        name.add("Rahul");
        name.add("Ganesh");
    }
}
```

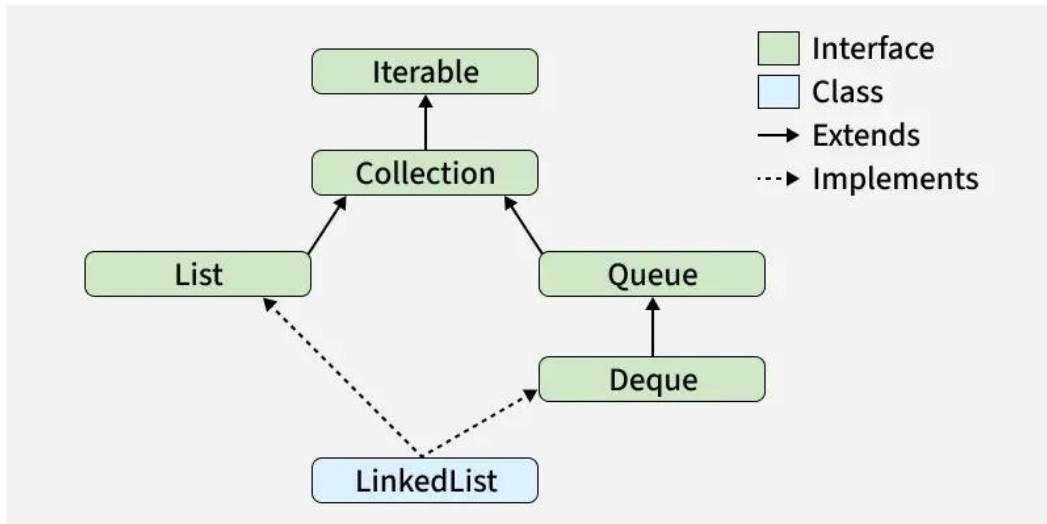
```
System.out.println("Generic Collection is: "+name);  
name.sort(null);  
System.out.println("After sorting is: "+name);  
  
Iterator it=name.iterator();  
  
while(it.hasNext())  
{  
    System.out.println(it.next());  
    Thread.sleep(1000);  
}  
  
}
```

Output :-

```
Generic Collection is: [Sagar, Rahul, Ganesh]  
After sorting is: [Ganesh, Rahul, Sagar]  
Ganesh  
Rahul  
Sagar
```

LinkedList in Java :-

- LinkedList is a class in the Java Collection Framework.
- It is present in the package java.util.
- It implements the List and Deque interfaces.
- It allows duplicate elements and maintains insertion order.
- It uses a doubly linked list structure internally.



Syntax :-

```
LinkedList<DataType> obj = new LinkedList<DataType>();
```

Mehods of LinkedList :-

Method	Description
add(E e)	Adds an element at the end of the list.
addFirst(E e)	Adds an element at the beginning.
addLast(E e)	Adds an element at the end.
get(int index)	Returns element at the specified index.
remove(int index)	Removes element at the given index.
removeFirst()	Removes the first element.
removeLast()	Removes the last element.
size()	Returns number of elements.
clear()	Removes all elements.

Example :-

```
package collections;

import java.util.Iterator;
import java.util.LinkedList;

public class LinkedList_demo {

    public static void main(String[] args) throws InterruptedException{
        LinkedList l=new LinkedList();

        l.add("abc@45gmail.com");
        l.add(678);
        l.add("Sony");
        l.add('#');

        System.out.println(l.offer(76));
        l.add(89.6);
        System.out.println("peek: "+l.peek());
        System.out.println("poll: "+l.poll());
        System.out.println("LinkedList: "+l);

        Iterator it=l.iterator();

        while(it.hasNext())
        {
            System.out.println(it.next());
            Thread.sleep(1000);
        }

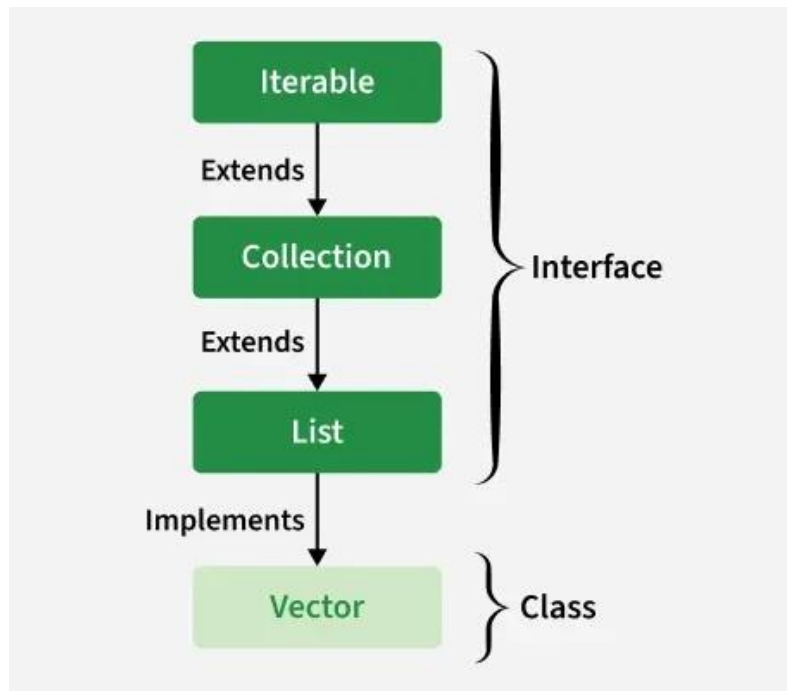
    }
}
```

Output :-

```
true
peek: abc@45gmail.com
poll: abc@45gmail.com
LinkedList: [678, Sony, #, 76, 89.6]
678
Sony
#
76
89.6
```

Vector class in Java :-

- Vector is a class in the Java Collection Framework.
- It is present in the package java.util.
- Vector implements the List interface.
- It is similar to ArrayList, but synchronized (thread-safe).
- It allows duplicate elements and maintains insertion order.



Syntax :-

`Vector<DataType> obj = new Vector<DataType>();`

Methods of Vector class :-

Method	Description
<code>add(E e)</code>	Adds an element.
<code>addElement(E e)</code>	Adds an element (old method).
<code>get(int index)</code>	Returns element at given index.
<code>remove(int index)</code>	Removes element at given index.
<code>size()</code>	Returns number of elements.
<code>capacity()</code>	Returns current capacity of vector.
<code>clear()</code>	Removes all elements.

Example :-

```
package collections;

import java.util.Iterator;
import java.util.Vector;

public class vector_demo {

    public static void main(String[] args) {
        //Generic Collection
        Vector<Integer>roll_no=new Vector<Integer>();
        roll_no.add(30);
        roll_no.add(50);
        roll_no.add(80);

        //Iterate the objects of collection change the value of index 1 to 60
        System.out.println("Vector elements after update:");
        roll_no.set(1,60);
        for(Integer i:roll_no)
        {
            System.out.println(i);
        }
    }
}
```

Output :-

```
Vector elements after update:
30
60
80
```

Difference Between ArrayList, LinkedList & Vector :-

Subject	Array List	Linked List	Vector
synchronized	Not present	Not present	present
Random access	Allowed	Not Allowed	Allowed
Memory Location	contiguous	Not contiguous	contiguous
Null values	supports	supports	supports
Data structure	Dynamic Array	Doubly Linked List	Dynamic Array
Duplicate allowed	Yes	Yes	Yes
Operation	Insertion and deletion are slow	Insertion and deletion are fast	Insertion and deletion are slow